

```

const PASSWORD = 'charlotte';
const SESSION_KEY = 'charlotte_session';
const PROGRESS_KEY = 'charlotte_progress';
const TODO_KEY = 'charlotte_todo';
const TODO_CATS_KEY = 'charlotte_todo_cats';

// LOGIN STARFIELD
const lc=document.getElementById('login-canvas'),lx=lc.getContext('2d');let lst=[
function rsz2(){lc.width=innerWidth;lc.height=innerHeight;}
function mkLStars(){lst=[];for(let i=0;i<160;i++){const r=Math.random()*0.9+0.1;1
function drawL(){lx.clearRect(0,0,lc.width,lc.height);lst.forEach(s=>{s.a+=s.sp;c
rsz2();mkLStars();drawL();window.addEventListener('resize',()=>{rsz2();mkLStars()

// LOGIN
const loginPage=document.getElementById('login-page'),pwInput=document.getElement
function unlock(){const val=pwInput.value.trim().toLowerCase();if(val===PASSWORD)
pwBtn.addEventListener('click',unlock);pwInput.addEventListener('keydown',e=>{if(
if(sessionStorage.getItem(SESSION_KEY)=== '1'){loginPage.classList.add('hidden');i

function initApp(){
  let progress={c:[],r:[]};
  try{const s=localStorage.getItem(PROGRESS_KEY);if(s)progress=JSON.parse(s);}cat
  function saveProgress(){localStorage.setItem(PROGRESS_KEY,JSON.stringify(progre

// COMPTE-JOURS
const start = new Date('2025-12-31');
const today = new Date();
const days = Math.floor((today - start) / (1000 * 60 * 60 * 24));
const dayEl = document.getElementById('day-count');
let current = 0;
const duration = 1800;
const step = days / (duration / 16);
function animateDays() {
  current += step;
  if(current >= days){ dayEl.textContent = days; return; }
  dayEl.textContent = Math.floor(current);
  requestAnimationFrame(animateDays);
}
setTimeout(animateDays, 600);

// — CAPSULE TEMPORELLE —
// Change CAPSULE_DATE pour fixer la date d'ouverture (format: 'YYYY-MM-DD')
// Change CAPSULE_MESSAGE pour écrire ton message
const CAPSULE_DATE = '2026-12-31';

```

```
const CAPSULE_MESSAGE = `Charlotte,
```

C'est le 31 décembre. Un an – presque exactement – depuis ce soir où tu es entrée

Je t'écris depuis un endroit où je ne sais pas encore ce que cette année va nous

Alors laisse-moi te dire ce que je sais.

Je sais que tu es la personne la plus étonnante que j'aie rencontrée. Pas de façon

Je sais que tu m'as changée – sans le vouloir, sans le savoir peut-être. Il y a d

Je sais que nos trajets en voiture comptent parmi mes moments préférés. Que j'aim

Je sais que tu mérites tout ce que je t'ai dit, tout ce que je t'ai écrit dans ce

Il y a une étoile quelque part dans la constellation de la Lyre qui porte ton nom

Si tu lis ces mots ce soir, c'est qu'on a tenu – toi et moi, tout ce qu'on est, t

Bonne année, Charlotte.

Ma fleur. Ma yasmina. Mon étoile.

Je t'aime encore plus qu'il y a un an.

Et je compte bien continuer longtemps.`;

```
const capsuleTarget = new Date(CAPSULE_DATE);
capsuleTarget.setHours(22,22,0,0);
const now = new Date();
const capsuleLocked = document.getElementById('capsule-locked');
const capsuleOpen    = document.getElementById('capsule-open');
const capsuleCD       = document.getElementById('capsule-countdown');
const capsuleMsg      = document.getElementById('capsule-message');

function updateCapsule() {
  const now2 = new Date();
  const diff = capsuleTarget - now2;
  if(diff <= 0) {
    capsuleLocked.style.display = 'none';
    capsuleOpen.style.display = 'block';
    capsuleMsg.innerHTML = CAPSULE_MESSAGE.replace(/\n/g, '<br>');
  } else {
    const d = Math.floor(diff / (1000*60*60*24));
    const h = Math.floor((diff % (1000*60*60*24)) / (1000*60*60));
    const m = Math.floor((diff % (1000*60*60)) / (1000*60));
    const s = Math.floor((diff % (1000*60)) / 1000);
```

```

        capsuleCD.textContent = d + 'j ' + String(h).padStart(2,'0') + 'h' + String
    }
}
updateCapsule();
setInterval(updateCapsule, 1000);

// STARFIELDS OVERLAY (todo + lettres)
function makeOverlayStars(canvasId) {
    const c = document.getElementById(canvasId);
    if(!c) return;
    const x = c.getContext('2d');
    let st = [];
    function r() { c.width = innerWidth; c.height = innerHeight; }
    function mk() {
        st = [];
        for(let i=0;i<200;i++){
            const layer=Math.random()<0.7?0:Math.random()<0.7?1:2;
            const rad=layer===0?Math.random()*0.35+0.1:layer===1?Math.random()*0.55+0
            const maxA=layer===0?0.22:layer===1?0.42:0.65;
            st.push({x:Math.random()*c.width,y:Math.random()*c.height,r:rad,a:Math.ra
        }
    }
    function draw() {
        x.clearRect(0,0,c.width,c.height);
        st.forEach(s=>{s.a+=s.sp;const al=(Math.sin(s.a)*0.5+0.5)*s.maxA+0.02;x.beg
        requestAnimationFrame(draw);
    }
    r(); mk(); draw();
    window.addEventListener('resize', ()=>{r(); mk();});
}
makeOverlayStars('todo-starfield');
makeOverlayStars('lettres-starfield');

// STARFIELD
const cv=document.getElementById('starfield'),cx=cv.getContext('2d');let ST=[];
function rsz(){cv.width=innerWidth;cv.height=Math.max(document.body.scrollHeigh
function mkStars(){ST=[];const n=Math.floor(cv.height/2.8);for(let i=0;i<n;i++)

// Suivi souris / doigt
let mx = -9999, my = -9999;
window.addEventListener('mousemove', e => { mx = e.clientX; my = e.clientY + wi
window.addEventListener('touchmove', e => {
    if(e.touches[0]){ mx = e.touches[0].clientX; my = e.touches[0].clientY + wind
}, { passive: true });
window.addEventListener('touchend', () => { mx = -9999; my = -9999; });
window.addEventListener('mouseleave', () => { mx = -9999; my = -9999; });

```

```

function draw(){
  cx.clearRect(0,0,cv.width,cv.height);
  ST.forEach(s=>{
    s.a += s.sp;
    // Distance au curseur
    const dx = s.x - mx, dy = s.y - my;
    const dist = Math.sqrt(dx*dx + dy*dy);
    const radius = 120;
    if(dist < radius && dist > 0){
      const force = (radius - dist) / radius * 0.6;
      s.ox += (dx/dist) * force;
      s.oy += (dy/dist) * force;
    }
    // Retour progressif à la position d'origine
    s.ox *= 0.92;
    s.oy *= 0.92;
    const al=(Math.sin(s.a)*0.5+0.5)*s.maxA+0.02;
    cx.beginPath();
    cx.arc(s.x + s.ox, s.y + s.oy, s.r, 0, Math.PI*2);
    cx.fillStyle=`rgba(235,232,228,${al})`;
    cx.fill();
  });
  requestAnimationFrame(draw);
}
rsz();mkStars();draw();window.addEventListener('resize',()=>{rsz();mkStars();})

// MODAL
const ov=document.getElementById('overlay'),mtag=document.getElementById('m-tag')
function openModal(tag,text){mtag.textContent=tag;mtxt.textContent=text;ov.classList.add('open');document.body.style.overflow='hidden';}
function closeM(){ov.classList.remove('open');document.body.style.overflow='';}
mcl.addEventListener('click',closeM);ov.addEventListener('click',e=>{if(e.target.id==='m-tag')openModal(e.target.textContent,e.target.textContent);});

// 30 MOTS
const compliments=["Éthérée","Céleste","Séraphique","Lyrique","Nébuleuse","Vert",
const cgrid=document.getElementById('comp-grid'),ctr=document.getElementById('c')
for(let i=0;i<60;i++){const b=document.createElement('button');b.className='stars';b.textContent=cgrid.textContent+found.size+' / 60 découverts';
cgrid.appendChild(b);}

// 200 RAISONS
const raisons=["Ton rire qui remplit toute la pièce","La façon dont tu prononce",
const grid=document.getElementById('stars-grid'),ctr=document.getElementById('c')
for(let i=0;i<200;i++){const b=document.createElement('button');b.className='stars';b.textContent=grid.textContent+found.size+' / 200 découvertes';
grid.appendChild(b);}

// SURPRISE 200
const surpriseOverlay = document.getElementById('surprise-overlay');
const surpriseClose = document.getElementById('surprise-close');

```

```

const surpriseCanvas = document.getElementById('surprise-canvas');
const sc = surpriseCanvas.getContext('2d');
let shootingStars = [];

function showSurprise(){
  // Ferme le modal d'abord
  ov.classList.remove('open');
  document.body.style.overflow = 'hidden';
  surpriseOverlay.classList.add('open');
  surpriseCanvas.width = innerWidth;
  surpriseCanvas.height = innerHeight;
  shootingStars = [];
  for(let i=0;i<18;i++){
    setTimeout(()=>{
      shootingStars.push({
        x: Math.random()*innerWidth*0.8,
        y: Math.random()*innerHeight*0.4,
        len: Math.random()*120+60,
        speed: Math.random()*6+4,
        alpha: 1,
        angle: Math.PI/4 + (Math.random()-0.5)*0.3
      });
    }, i*180);
  }
  animateSurprise();
}

function animateSurprise(){
  if(!surpriseOverlay.classList.contains('open')) return;
  sc.clearRect(0,0,surpriseCanvas.width,surpriseCanvas.height);
  shootingStars.forEach((s,idx)=>{
    s.x += Math.cos(s.angle)*s.speed;
    s.y += Math.sin(s.angle)*s.speed;
    s.alpha -= 0.012;
    if(s.alpha <= 0) return;
    sc.save();
    sc.globalAlpha = s.alpha;
    const grad = sc.createLinearGradient(s.x,s.y,s.x-Math.cos(s.angle)*s.len,s.y-Math.sin(s.angle)*s.len);
    grad.addColorStop(0,'rgba(212,201,176,0.9)');
    grad.addColorStop(1,'rgba(212,201,176,0)');
    sc.strokeStyle = grad;
    sc.lineWidth = 1.2;
    sc.beginPath();
    sc.moveTo(s.x,s.y);
    sc.lineTo(s.x-Math.cos(s.angle)*s.len,s.y-Math.sin(s.angle)*s.len);
    sc.stroke();
    sc.restore();
  });
}

```

```

});
shootingStars = shootingStars.filter(s=>s.alpha>0);
// Relance des étoiles filantes en boucle
if(shootingStars.length < 3){
  shootingStars.push({
    x: Math.random()*innerWidth*0.7,
    y: Math.random()*innerHeight*0.35,
    len: Math.random()*120+60,
    speed: Math.random()*5+4,
    alpha: 1,
    angle: Math.PI/4+(Math.random()-0.5)*0.3
  });
}
requestAnimationFrame(animateSurprise);
}

surpriseClose.addEventListener('click', ()=>{
  surpriseOverlay.classList.remove('open');
  document.body.style.overflow='';
});

// — TODO —
// Categories de base dans le code
const baseCats = [
  { title: '✦ Random à faire', items: ["Meilleure librairie de Casa","Les autre
  { title: '✦ Manger', items: ["Glacier Baggli","Chez Aicha la vietnamienne","L
  { title: '✦ Side Quest', items: ["Écrire un son ensemble","Découvrir les fleu
  { title: '✦ Movies — aller beaucoup au cinéma', items: ["Labyrinthe","Reminde
  { title: '✦ Les exceptionnels', items: ["Saut en parachute à Benslimane"]},
  { title: '✦ Chez Charlotte', items: ["À compléter..."]}
];

// Charger les cats custom ajoutées par elle (stockées séparément)
let customCats = [];
try { const s = localStorage.getItem(TODO_CATS_KEY); if(s) customCats = JSON.pa
function saveCustomCats(){ localStorage.setItem(TODO_CATS_KEY, JSON.stringify(c

// État des cases cochées
let todoState = {};
try { const s = localStorage.getItem(TODO_KEY); if(s) todoState = JSON.parse(s)
function saveTodo(){ localStorage.setItem(TODO_KEY, JSON.stringify(todoState));

const TODO_DELETED_KEY = 'charlotte_todo_deleted';
let deletedItems = {};
try { const s = localStorage.getItem(TODO_DELETED_KEY); if(s) deletedItems = JS
function saveDeleted(){ localStorage.setItem(TODO_DELETED_KEY, JSON.stringify(d

```

```

const todoListEl = document.getElementById('todo-list');
const todoCount = document.getElementById('todo-count');
const todoFill = document.getElementById('todo-fill');
const todoPage = document.getElementById('todo-page');
const todoBackBtn = document.getElementById('todo-back');
const navTodoBtn = document.getElementById('nav-todo-btn');
let totalItems = 0;

function buildCategory(cat, ci, isCustom) {
  const catEl = document.createElement('div');
  catEl.className = 'todo-category';

  // Header avec titre + bouton +
  const header = document.createElement('div');
  header.className = 'todo-cat-header';
  const titleEl = document.createElement('p');
  titleEl.className = 'todo-category-title';
  titleEl.textContent = cat.title;
  const addBtn = document.createElement('button');
  addBtn.className = 'todo-add-item-btn';
  addBtn.textContent = '+ ajouter';
  header.appendChild(titleEl);
  header.appendChild(addBtn);
  // Delete category button (only for custom cats)
  if(isCustom){
    const delCatBtn = document.createElement('button');
    delCatBtn.className = 'todo-delete-cat';
    delCatBtn.textContent = 'x';
    delCatBtn.title = 'Supprimer la catégorie';
    delCatBtn.addEventListener('click', () => {
      if(!confirm('Supprimer cette catégorie ?')) return;
      const idx = customCats.indexOf(cat);
      if(idx > -1){ customCats.splice(idx, 1); saveCustomCats(); }
      // Remove all todo states for this cat
      Object.keys(todoState).forEach(k => { if(k.startsWith(ci+'-')) delete tod
      saveTodo();
      catEl.remove();
      totalItems = Math.max(0, totalItems - cat.items.length);
      updateProgress();
    });
    header.appendChild(delCatBtn);
  }
  catEl.appendChild(header);

  // Inline add form
  const addForm = document.createElement('div');
  addForm.className = 'todo-add-form';

```

```

const addInput = document.createElement('input');
addInput.className = 'todo-add-input';
addInput.type = 'text';
addInput.placeholder = 'Nouveau truc...';
const addConfirm = document.createElement('button');
addConfirm.className = 'todo-add-confirm';
addConfirm.textContent = 'Ajouter ✦';
addForm.appendChild(addInput);
addForm.appendChild(addConfirm);
catEl.appendChild(addForm);

addBtn.addEventListener('click', () => {
  addForm.classList.toggle('open');
  if(addForm.classList.contains('open')) addInput.focus();
});

function addItem(text) {
  text = text.trim();
  if(!text) return;
  cat.items.push(text);
  if(isCustom) saveCustomCats();
  const ii = cat.items.length - 1;
  const key = ci + '-' + ii;
  totalItems++;
  appendItem(catEl, addForm, text, key, true);
  updateProgress();
  addInput.value = '';
  addForm.classList.remove('open');
}

addConfirm.addEventListener('click', () => addItem(addInput.value));
addInput.addEventListener('keydown', e => { if(e.key === 'Enter') addItem(add

// Items existants
cat.items.forEach((item, ii) => {
  const key = ci + '-' + ii;
  if(deletedItems[key]) return; // skip deleted
  totalItems++;
  appendItem(catEl, addForm, item, key, true);
}));

todoListEl.appendChild(catEl);
return catEl;
}

function appendItem(catEl, addForm, item, key, isCustomItem) {
  const row = document.createElement('div');

```



```

row.className = 'todo-item' + (todoState[key] ? ' done' : '') + (isCustomItem
row.innerHTML = '<div class="todo-check"></div><span class="todo-label">' + i
// Delete button for custom items
if(isCustomItem){
  const delBtn = document.createElement('button');
  delBtn.className = 'todo-delete-item';
  delBtn.textContent = 'x';
  delBtn.title = 'Supprimer';
  delBtn.addEventListener('click', e => {
    e.stopPropagation();
    delete todoState[key];
    deletedItems[key] = true;
    saveTodo();
    saveDeleted();
    row.remove();
    totalItems = Math.max(0, totalItems - 1);
    updateProgress();
  });
  row.appendChild(delBtn);
}
row.addEventListener('click', () => {
  todoState[key] = !todoState[key];
  row.classList.toggle('done', todoState[key]);
  saveTodo(); updateProgress();
});
catEl.insertBefore(row, addForm);
}

// Build all base + custom categories
const allCats = [...baseCats, ...customCats];
allCats.forEach((cat, ci) => buildCategory(cat, ci, ci >= baseCats.length));

function updateProgress(){
  const done = Object.values(todoState).filter(Boolean).length;
  todoCount.textContent = done + ' / ' + totalItems + ' complétées';
  todoFill.style.width = totalItems ? (done / totalItems * 100) + '%' : '0%';
}
updateProgress();

// ON FAIT QUOI
const pickerBtn = document.getElementById('todo-picker-btn');
const pickerResult = document.getElementById('todo-picker-result');

pickerBtn.addEventListener('click', () => {
  // Collecte tous les items non cochés et non supprimés
  const available = [];
  allCats.forEach((cat, ci) => {

```

```

    cat.items.forEach((item, ii) => {
      const key = ci + '-' + ii;
      if(!todoState[key] && !deletedItems[key]) available.push(item);
    });
  });
  if(available.length === 0){
    pickerResult.textContent = 'On a tout fait !';
    return;
  }
  pickerResult.style.opacity = '0';
  setTimeout(() => {
    const pick = available[Math.floor(Math.random() * available.length)];
    pickerResult.textContent = pick;
    pickerResult.style.opacity = '1';
  }, 200);
});

// Nouvelle catégorie
const newCatBtn = document.getElementById('new-cat-btn');
const newCatForm = document.getElementById('new-cat-form');
const newCatInput = document.getElementById('new-cat-input');
const newCatConfirm = document.getElementById('new-cat-confirm');

newCatBtn.addEventListener('click', () => {
  newCatForm.classList.toggle('open');
  if(newCatForm.classList.contains('open')) newCatInput.focus();
});

function addCategory() {
  const name = newCatInput.value.trim();
  if(!name) return;
  const newCat = { title: '✦ ' + name, items: [] };
  customCats.push(newCat);
  saveCustomCats();
  const ci = allCats.length;
  allCats.push(newCat);
  buildCategory(newCat, ci, true);
  newCatInput.value = '';
  newCatForm.classList.remove('open');
}

newCatConfirm.addEventListener('click', addCategory);
newCatInput.addEventListener('keydown', e => { if(e.key === 'Enter') addCategor

// Ouvrir / fermer todo
navTodoBtn.addEventListener('click', () => {
  todoPage.classList.add('open');
  navTodoBtn.classList.add('active');

```

```

    document.body.style.overflow = 'hidden';
  });
  todoBackBtn.addEventListener('click', () => {
    todoPage.classList.remove('open');
    navTodoBtn.classList.remove('active');
    document.body.style.overflow = '';
    document.getElementById('todo-inner').scrollTop = 0;
  });

  // — LETTRES —
  const LETTRES_KEY = 'charlotte_lettres';

  // =====
  // AJOUTER LETTRES ICI
  // Pour chaque lettre : { title: "Lettre n° X", text: `texte ici` }
  const lettres = [
    {
      title: "Lettre n° 1",
      text: `Charlotte,

```

Cette lettre est la première d'une longue série — du moins, c'est ce que je me pr

Il y a des choses que je n'arrive pas toujours à te dire de vive voix. Pas parce

Je ne sais pas exactement ce que je t'écirai. Peut-être des choses que j'observe

Considère cet endroit comme ma façon de continuer à te choisir, même dans le sile  
 },

```

  {
    title: "Lettre n° 2 - 13/07/2026",
    text: `Charlotte,

```

Aujourd'hui, il ne s'est rien passé d'extraordinaire. Tu m'as juste un peu beauc

On croit souvent que les grandes déclarations naissent des grands moments. Pourta

J'aime apprendre ces détails-là de toi.

J'aime découvrir la personne que tu es dans les gestes auxquels tu ne fais même p

J'aime penser qu'on passe notre vie à connaître quelqu'un sans jamais avoir fini

Alors aujourd'hui, je voulais simplement laisser une trace de cette pensée. J'aim

JE T'AIME. Et en même jour, le 13 février, on s'embrassait pour la première fois  
 },

```

// — LETTRE 3 —
// {
//   title: "Lettre n° 3",
//   text: `texte ici`
// },
];
// =====

let lettresOpened = {};
try { const s = localStorage.getItem(LETTRES_KEY); if(s) lettresOpened = JSON.p
function saveLettres(){ localStorage.setItem(LETTRES_KEY, JSON.stringify(lettre

const envGrid = document.getElementById('envelopes-grid');
const lettreOverlay = document.getElementById('lettre-overlay');
const lettreNum = document.getElementById('lettre-num');
const lettreContent = document.getElementById('lettre-content');
const lettreClose = document.getElementById('lettre-close');

lettres.forEach((l, i) => {
  const btn = document.createElement('button');
  btn.className = 'envelope-btn' + (lettresOpened[i] ? ' opened' : '');
  btn.innerHTML = `<span class="envelope-icon">✦</span><span class="envelope-nu
  btn.addEventListener('click', () => {
    if(!lettresOpened[i]){
      lettresOpened[i] = true;
      btn.classList.add('opened');
      saveLettres();
    }
    lettreNum.textContent = l.title;
    lettreContent.textContent = l.text;
    lettreOverlay.classList.add('open');
    document.body.style.overflow = 'hidden';
  });
  envGrid.appendChild(btn);
});

function closeLettreModal(){ lettreOverlay.classList.remove('open'); document.b
lettreClose.addEventListener('click', closeLettreModal);
lettreOverlay.addEventListener('click', e => { if(e.target === lettreOverlay) c

const navLettresBtn = document.getElementById('nav-lettres-btn');
const lettresPage = document.getElementById('lettres-page');
const lettresBack = document.getElementById('lettres-back');

navLettresBtn.addEventListener('click', () => {
  lettresPage.classList.add('open');
  navLettresBtn.classList.add('active');

```

```

    document.body.style.overflow = 'hidden';
  });
  lettresBack.addEventListener('click', () => {
    lettresPage.classList.remove('open');
    navLettresBtn.classList.remove('active');
    document.body.style.overflow = '';
    document.getElementById('lettres-inner').scrollTop = 0;
  });

// — SON AMBIANT —
const audio = new Audio('birds.mp3');
audio.loop = true;
audio.volume = 0.25;
const soundBtn = document.getElementById('nav-sound-btn');
let soundOn = false;
soundBtn.addEventListener('click', () => {
  soundOn = !soundOn;
  if(soundOn){
    audio.play();
    soundBtn.textContent = 'ON';
    soundBtn.classList.add('on');
  } else {
    audio.pause();
    soundBtn.textContent = 'OFF';
    soundBtn.classList.remove('on');
  }
});

// — GALERIE —
// Pour ajouter des photos : ajoute simplement "album21.png", "album22.png"...
const photos = [
  'album1.png', 'album2.png', 'album3.png', 'album4.png', 'album5.png',
  'album6.png', 'album7.png', 'album8.png', 'album9.png', 'album10.png',
  'album11.png', 'album12.png', 'album13.png', 'album14.png', 'album15.png',
  'album16.png', 'album17.png', 'album18.png', 'album19.png', 'album20.png'
];

const galleryGrid = document.getElementById('gallery-grid');
const lightbox = document.getElementById('lightbox');
const lbImg = document.getElementById('lightbox-img');
const lbClose = document.getElementById('lightbox-close');
const lbPrev = document.getElementById('lightbox-prev');
const lbNext = document.getElementById('lightbox-next');
let currentPhoto = 0;

photos.forEach((src, i) => {
  const item = document.createElement('div');

```

```

    item.className = 'gallery-item';
    const img = document.createElement('img');
    img.src = src;
    img.alt = 'Charlotte & Lily ' + (i + 1);
    img.loading = 'lazy';
    item.appendChild(img);
    item.addEventListener('click', () => openLightbox(i));
    galleryGrid.appendChild(item);
  });

function openLightbox(i) {
  currentPhoto = i;
  lbImg.src = photos[i];
  lightbox.classList.add('open');
  document.body.style.overflow = 'hidden';
}
function closeLightbox() {
  lightbox.classList.remove('open');
  document.body.style.overflow = '';
}
function showPhoto(i) {
  currentPhoto = (i + photos.length) % photos.length;
  lbImg.src = photos[currentPhoto];
}

lbClose.addEventListener('click', closeLightbox);
lightbox.addEventListener('click', e => { if(e.target === lightbox) closeLightb
lbPrev.addEventListener('click', e => { e.stopPropagation(); showPhoto(currentP
lbNext.addEventListener('click', e => { e.stopPropagation(); showPhoto(currentP
document.addEventListener('keydown', e => {
  if(!lightbox.classList.contains('open')) return;
  if(e.key === 'ArrowLeft') showPhoto(currentPhoto - 1);
  if(e.key === 'ArrowRight') showPhoto(currentPhoto + 1);
  if(e.key === 'Escape') closeLightbox();
});

} // end initApp

```